NoC Topology Characterization Under Diverse Traffic Workloads

Steve Cho
Department of Electrical and Computer Engineering
Carnegie Mellon University
Pittsburgh, PA, USA
stevecho@andrew.cmu.edu

Kushal Agrawal
Department of Electrical and Computer Engineering
Carnegie Mellon University
Pittsburgh, PA, USA
kushalag@andrew.cmu.edu

***Abstract*—Our project investigates how the structural properties of Network-on-Chip (NoC) topologies affect latency and throughput under different traffic workloads. We characterize the Mesh, Torus, Ring, and Tree topologies using graph analysis and simulate them on the gem5 Garnet simulator. Preliminary results have shown that latency at low injection rates is significantly related to path length, while under a high injection rate, performance is affected by bottleneck links and network saturation. We will generate a full set of predictions using graph analysis and simulation in all synthetic traffic patterns. We will look into modern NoC designs to apply our analysis to them.**

***Index Terms*—Network on Chip (NoC), Graph Analysis, gem5, Garnet**

I. Introduction and Motivation

Network-on-Chip (NoC) provides network-based communication that connects subsystems, such as cores and caches, in modern multicore processors. As the number of compute units increases, on-chip communication can become a major bottleneck, affecting latency and throughput. Thus, the NoC topology dictates how data travel between these subsystems. Furthermore, different traffic workloads generate distinct communication patterns and no single topology is best for all traffic workloads. Rather than focusing on conventional sources of performance limitations, such as memory access and instruction execution, we focus specifically on the communication network. In particular, our aim is to determine which structural properties of a NoC topology determine latency and throughput under different traffic workloads. Our approach to answering this question is to characterize Mesh, Torus, Ring, and Tree topologies using graph properties, such as average hops, diameter, connectivity, and betweenness, to predict how they will behave for each traffic pattern. We then plan to use gem5 [3] Garnet [4] to simulate under multiple synthetic traffic patterns and compare the measured performance with our predictions. We also plan to investigate some modern chips to see what topologies they use and the reasons behind their design choices.

II. Previous Work

Previous work has shown that the topology and traffic workload significantly affect NoC performance. Bakhoda et al. [1] studied on-chip networks for accelerators with a large number of cores and also showed that GPU traffic can create a communication bottleneck. This is mainly because many cores send requests to a small number of memory controllers, and the responses are large. As a result, traffic is concentrated near the memory controller nodes, creating bottlenecks in the network. Their work focuses mainly on the 2D mesh topology and the many-to-few-to-many traffic pattern. Pande et al. [2] directly explore throughput and latency across different topologies under varying traffic patterns. Their results show that highly connected topologies perform well across most uniform traffic, while less connected topologies perform well under localized traffic.

Our work builds on these works and focuses on analysis using the structural properties of the network itself. Instead of relying solely on measured performance, we use graph metrics such as average path length, diameter, connectivity, and betweenness to explain why certain topologies perform well or poorly under specific traffic workloads. We then compare these graph-based predictions with the simulated results using gem5 Garnet.

III. Approach

We model each NoC topology as a graph and use graph analysis to predict its latency and throughput. We then test these predictions with gem5/Garnet [3], [4], a simulator that models the network one clock cycle at a time and tracks every packet as it moves between routers. If a prediction matches the simulation, the graph structure alone explains that topology's performance. If it does not match, the network is limited by something the graph does not model, such as the internal behavior of the routers.

A. Network Construction

Nodes and links. Cores in a NoC do not talk to each other directly [5]. Each core connects to a small switch, called a *router*, that passes packets toward their destination, so we use routers as the nodes of our graph. In the Mesh, Torus, and Ring, each of the 16 routers has one core attached. In the Tree, the 16 cores sit at the leaves, and the 15 routers above them only forward traffic. Edges are the wires between routers. Each wire works in both directions and carries one packet per cycle, taking one cycle to cross.

TABLE I
STRUCTURAL PROPERTIES OF THE 16-CORE TOPOLOGIES

	Mesh	Torus	Ring	Tree
Routers	16	16	16	31
Links	24	32	16	30
Router degree	2–4	4	2	1–3
Diameter (hops)	6	4	8	8
Avg. hops (core pairs)	2.67	2.13	4.27	6.53
Bisection width (links)	4	8	2	1

Topologies. The topology is the pattern in which the routers are wired together. We compare four common 16-core topologies (Fig. 1). The *Mesh* is a 4×4 grid where each router connects to its up, down, left, and right neighbors. The *Torus* is the same grid with extra links that wrap around from each edge to the opposite edge, so no router sits at a border. The *Ring* connects all 16 routers in a single circle. The *Tree* is a hierarchy in which traffic between two cores climbs up toward the root and back down. *gem5 provides only the Mesh, so we implemented the other three ourselves*. Table I lists their graph properties.

Routing. The routing rule decides which path a packet takes when more than one is possible. In all four topologies, packets always take a shortest path, the most common choice in real NoCs [5], [8]. In Mesh and Torus, a packet first travels horizontally until it reaches the destination's column, then vertically. In the Ring, it goes whichever way around the circle is shorter. When both directions are equally short (in the Ring or across the Torus), the simulator picks one at random. In the Tree there is only one path: up to the first router the two cores share, then back down.

Input data (Traffic). Our input is synthetic traffic created by gem5's built-in traffic generator [3], [4], controlled by two settings. The *traffic pattern* decides who sends to whom. For example, in *uniform random* each packet goes to a randomly chosen core; in *transpose* the core at position (x, y) always sends to (y, x); in *bit-complement* each core sends to the core on the opposite side of the chip; and in *neighbor* each core sends to the core next to it. We use all eight built-in patterns (the other four are bit-reverse, bit-rotation, shuffle, and tornado). The *injection rate* λ sets how busy the network is: it is the chance that a core sends a packet in a given cycle, so $\lambda = 0.1$ means about one packet every 10 cycles. We use synthetic traffic because each pattern stresses the network in one specific way. Real programs mix many patterns together, which makes it hard to tell which one caused a slowdown.

Assumptions. We make a few simplifications so that performance differences come from the topology itself:

- Traffic is synthetic: each core independently sends a packet with probability λ each cycle.
- All packets are the same size (one flit).
- Every link takes one cycle to cross. In a real chip, long wires (Torus wrap-arounds, Tree links near the root) would be slower; we ignore that so a topology is not punished just for having longer wires.
- Every router is identical: one cycle of delay and four virtual channels (separate buffer lanes on each link). We do not give some topologies faster routers or more buffers than others.
- Who sends to whom is the same for every topology. Core 0 always sends to the same destination under a given pattern, whether the wiring is a Mesh, Torus, Ring, or Tree. That way we compare how each topology carries the same traffic, not different traffic on each one.

B. Analysis and Research Questions

A topology's performance looks different at low and high load, so we split the question into three parts. At low load, packets rarely wait behind each other, so the main cost should be how many hops they travel. At high load, waiting on busy links dominates, so the main limit should be the single busiest link. Across both regimes, we want to know which graph properties drive the differences, and when the graph is not enough. This leads to three research questions:

- **RQ1 (low load):** When the network is lightly loaded, does average hop count fully explain packet latency? If yes, a shorter-path topology (like the Torus) should always be faster than a longer-path one (like the Tree) under the same traffic.
- **RQ2 (high load):** When the network is heavily loaded, can we predict the injection rate at which it saturates by finding its busiest link under the routing rule and traffic pattern in use? If yes, graph analysis can forecast throughput without a full simulation.
- **RQ3 (why/when) (Method TBD):** Which structural properties (hops, bisection width, betweenness) explain why a topology wins under one traffic pattern but loses under another, where does the graph model fail (for example, deadlock), and how do these findings relate to topologies used in commercial processors and accelerators?

RQ1. We use the standard zero-load latency model from Dally and Towles [5]:

$$T_0 = H_{\text{avg}}\,(t_r + t_w) + T_{\text{fixed}}, \tag{1}$$

where H_{avg} is the average hop count from the graph, t_r and t_w are the per-hop router and link delays, and T_{fixed} is the cost of entering and leaving the network. If RQ1 holds, T_0 plotted against H_{avg} should be a straight line across all topologies and patterns.

RQ2. The network saturates when its busiest link fills up [5]. To find that link, we pretend every core sends one packet per cycle, follow each packet's path under our routing rule, and count how many packets use each link. Call that count γ_ℓ. The predicted saturation rate is then simply one over the largest count:

$$\Theta^* = \min\left(1,\ \frac{1}{\max_\ell \gamma_\ell}\right). \tag{2}$$

(The cap of 1 is because a core cannot send more than one packet per cycle.) For example, take bit-complement traffic: each core sends to the core on the opposite side of the chip.

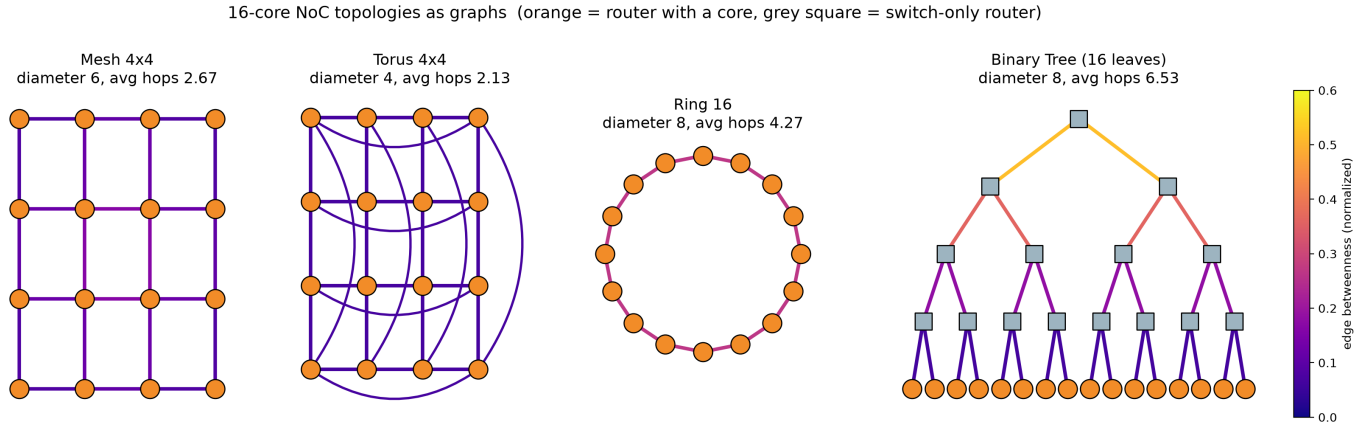


Fig. 1. The four 16-core topologies as graphs. Orange nodes are routers with an attached core; grey squares are forwarding-only routers. Link color shows normalized edge betweenness. In the Tree, the links near the root carry most shortest paths and are the expected bottleneck.

On the Tree, every packet that leaves the left half must climb through the single root link, so 8 packets share that one link and $\Theta^* = 1/8 = 0.125$. On the Mesh, the same left-to-right traffic can spread across the 4 vertical cuts in the middle of the grid, so each of those links sees only 2 packets and $\Theta^* = 1/2 = 0.5$. The Mesh can therefore handle four times the load of the Tree under this pattern.

IV. Experimental Setup and Results

A. Experimental Setup

We run all experiments in gem5 with its Garnet network model [3], [4], which simulates the NoC one clock cycle at a time. Each of the 16 cores sends synthetic traffic; the destination just receives the packets. All routers and links take one cycle, packets are one flit long, and each link has four virtual channels (separate buffer lanes on the same wire, used so one waiting packet does not block the whole link; not part of the topology graph).

The pieces fit together as follows. Each topology is a small Python file that tells gem5 how to wire the routers. The same wiring is also read as a graph in NetworkX [7], which computes hop counts and the busiest-link prediction Θ^*. A sweep script then launches gem5 for every topology, traffic pattern, and injection rate, collects the results, and we compare those measurements to the graph predictions. We test every combination of the 4 topologies, 8 traffic patterns, and 32 injection rates ($\lambda = 0.025$ to 0.800), for 1,024 runs in total. For each run we record average latency, average hops, and how many packets were sent and received.

We report two main quantities. *Low-load latency* T_0 is how long a packet takes when the network is nearly empty (our smallest injection rate). That is the RQ1 measurement. As we raise the injection rate, latency stays flat for a while and then shoots up: that jump is *saturation*, the highest load the network can handle before it clogs. In practice we mark saturation at the first rate where latency exceeds $3T_0$, or where fewer than 95% of sent packets arrive. That measured rate is what we compare to the predicted Θ^* from RQ2.

To check that gem5 built the same networks we analyzed as graphs, we compare average hops under uniform traffic to Table I. They match in rank order and stay within about 7%, so the simulated wiring is consistent with the graph (Sanity check).

B. Preliminary Results

Fig. 2 and Table II summarize all 1,024 runs.

a) RQ1 (low load): When the network is nearly empty, hop count fully predicts latency. Fig. 2(B) plots T_0 against average hops for every topology and traffic pattern: all 32 points fall on one line,

$$T_0 = 2.01\, H_{\text{avg}} + 4.98 \quad (R^2 = 1.000).$$

The slope of ≈ 2 is exactly our one-cycle router plus one-cycle link. The intercept of ≈ 5 is the fixed cost of entering and leaving the network. So a shorter-path topology is always faster at low load: Torus wins, Tree loses.

b) RQ2 (high load): Fig. 2(C) and Table II (excluding Ring deadlock) show a mean absolute error of about 0.08 ($\approx$10% relative to Θ^*). Tree and Mesh bit-complement match exactly (0.125 and 0.50). The largest misses are $\Theta^* = 1$ cases (e.g., Mesh uniform 1.00 vs. 0.63), due to router contention below.

Under uniform traffic (Fig. 2(A)), the networks clog in this order: Tree (0.225), Ring (0.275, deadlock), Mesh (0.625), Torus (0.75). That is the same order as bisection width in Table I (1, 2, 4, 8 links). Which topology is best also depends on the traffic: for nearby (neighbor) traffic, Mesh and Torus tie at 0.80; for cross-chip (bit-complement) traffic, Torus handles about $1.6\times$ more load because wrap-around links give packets a shorter path across the chip.

c) Where the graph falls short: Two effects are outside the graph model:

- **Router contention.** For Mesh and Torus under uniform traffic, the model predicts a limit of 1.0, but gem5 saturates at 0.63 and 0.75. Packets fight for buffers and the router switch; that waste is not in the graph.



Fig. 2. Preliminary results (16 cores, single-flit packets). (A) Average latency versus injection rate under uniform random traffic; each curve ends where the network saturates. (B) Low-load latency versus average hop count for all 32 topology–pattern combinations. (C) Saturation rate predicted by the busiest-link model (Eq. 2) versus the rate measured in gem5; points on the dashed line are predicted exactly.

TABLE II
PREDICTED / SIMULATED SATURATION INJECTION RATE (PACKETS/CORE/CYCLE). HIGHER IS BETTER.

	Uniform	**Transpose**	**Bit-compl.**	**Bit-rev.**	**Bit-rot.**	**Shuffle**	**Neighbor**‡
Mesh	1.00 / 0.63	0.33 / 0.35	0.50 / 0.50	0.33 / 0.35	0.50 / 0.50	0.50 / 0.50	1.00 / 0.80
Torus	1.00 / 0.75	0.67 / 0.60	1.00 / 0.80	0.67 / 0.55	0.67 / 0.63	0.67 / 0.65	1.00 / 0.80
Ring	0.50 / 0.28†	0.33 / 0.30	0.25 / 0.25	0.33 / 0.30	0.25 / 0.25	0.25 / 0.25	1.00 / 0.80
Tree	0.25 / 0.23	0.25 / 0.23	0.13 / 0.13	0.25 / 0.23	0.25 / 0.23	0.25 / 0.25	1.00 / 0.80

† Deadlock.
‡ With 16 cores, tornado produces the same destination mapping as neighbor, resulting in identical results.
Note: Simulated saturation rates are measured at injection-rate increments of 0.025.

- **Deadlock.** For the Ring under uniform traffic, the model predicts 0.50, but the network freezes at 0.275 and almost no packets arrive. This behavior suggests a deadlock behavior, where packets going both ways around the circle wait on each other forever. A standard fix is to split the Ring's buffers into virtual-channel classes at a dateline [6], [8]; adding that is a next step.

d) Takeaway: Overall, treating the NoC as a graph works surprisingly well. When traffic is light, hop count tells you the latency. When traffic is heavy, the busiest link tells you when the network will clog. The failures are useful too. Router contention and Ring deadlock show what the graph cannot capture, and those are the issues we plan to look at next.

V. CONCLUSION AND SHORT-TERM PLANS

Our project focuses on the impact of structural properties and traffic patterns on NoC's latency and throughput. Our preliminary results showed that at low injection rates, latency is significantly correlated with path length, while at high injection rates, the busiest links fill up, and the network saturates.

For the final presentation, we plan to perform any additional set of simulations of throughput and latency as needed and further analyze the relationship between graph properties and measured performance. We also plan to investigate the type of NoC topology and the design choices of modern processors and accelerators. In addition, we need to resolve the Ring (and Torus) deadlock and re-running those cases. Steve will continue developing the graph-metric and stats.txt parser script and focus on Mesh and Torus topology analysis. Kushal will continue to develop the sweep script and Ring and Tree topology analysis. Both members will work together on the final comparison, real-world case study, presentation, and report.

REFERENCES

[1] A. Bakhoda, J. Kim, and T. M. Aamodt, "Throughput-effective on-chip networks for manycore accelerators," in *Proc. 43rd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2010, pp. 421–432, doi: 10.1109/MICRO.2010.50.

[2] P. P. Pande, C. Grecu, M. Jones, A. Ivanov, and R. Saleh, "Performance evaluation and design trade-offs for network-on-chip interconnect architectures," *IEEE Trans. Comput.*, vol. 54, no. 8, pp. 1025–1040, 2005, doi: 10.1109/TC.2005.134.

[3] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 simulator," *ACM SIGARCH Comput. Archit. News*, vol. 39, no. 2, pp. 1–7, 2011, doi: 10.1145/2024716.2024718.

[4] N. Agarwal, T. Krishna, L.-S. Peh, and N. K. Jha, "GARNET: A detailed on-chip network model inside a full-system simulator," in *Proc. IEEE Int. Symp. Performance Analysis of Systems and Software (ISPASS)*, 2009, pp. 33–42, doi: 10.1109/ISPASS.2009.4919636.

[5] W. J. Dally and B. Towles, *Principles and Practices of Interconnection Networks*. San Francisco, CA, USA: Morgan Kaufmann, 2004.

[6] W. J. Dally and C. L. Seitz, "Deadlock-free message routing in multiprocessor interconnection networks," *IEEE Trans. Comput.*, vol. C-36, no. 5, pp. 547–553, May 1987, doi: 10.1109/TC.1987.1676939.

[7] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python in Science Conf. (SciPy)*, 2008, pp. 11–15.

[8] N. Enright Jerger, T. Krishna, and L.-S. Peh, *On-Chip Networks*, 2nd ed., ser. Synthesis Lectures on Computer Architecture. San Rafael, CA, USA: Morgan & Claypool, 2017.